



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Dipartimento di Scienze Fisiche,
Informatiche e Matematiche

9. Ottimizzazioni sui Loop

Linguaggi e Compilatori [I215-011]

Corso di Laurea in INFORMATICA
(D.M.270/04) [16-262]
Anno accademico 2022/2023

Prof. Andrea Marongiu
andrea.marongiu@unimore.it

Copyright note

È vietata la copia e la riproduzione dei contenuti e immagini in qualsiasi forma.

È inoltre vietata la redistribuzione e la pubblicazione dei contenuti e immagini non autorizzata espressamente dall'autore o dall'Università di Modena e Reggio Emilia.

Credits

- Cooper, Torczon, “Engineering a Compiler”, Elsevier
- Sampson, Cornell University, “Advanced Compilers”
- Gibbons, Carnegie Mellon University, “Optimizing Compilers”
- Pekhimenko, University of Toronto, “Compiler Optimization”

Outline

- Loop-Invariant Code Motion
- Induction Variable Elimination (Strength Reduction)



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Dipartimento di Scienze Fisiche,
Informatiche e Matematiche

Loop-Invariant Code Motion

Loop-Invariant Code Motion

```
a = ...;  
b = ...;  
for (i = 0; i < 100; i++)  
    f(a * b);
```

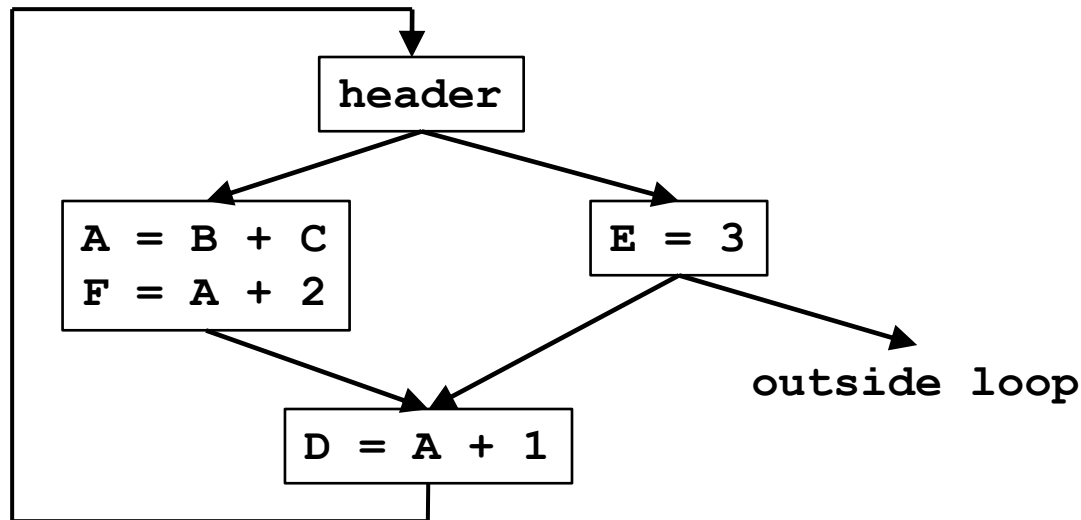


```
a = ...;  
b = ...;  
c = a * b;  
for (i = 0; i < 100; i++)  
    f(c);
```

- Sposta le istruzioni che non dipendono dal control flow del loop fuori dal loop stesso
- Evita di ricalcolare in maniera ridondante la stessa cosa
- Essendo il grosso della computazione di un programma contenuta nei loop c'è enorme potenziale di miglioramento della performance

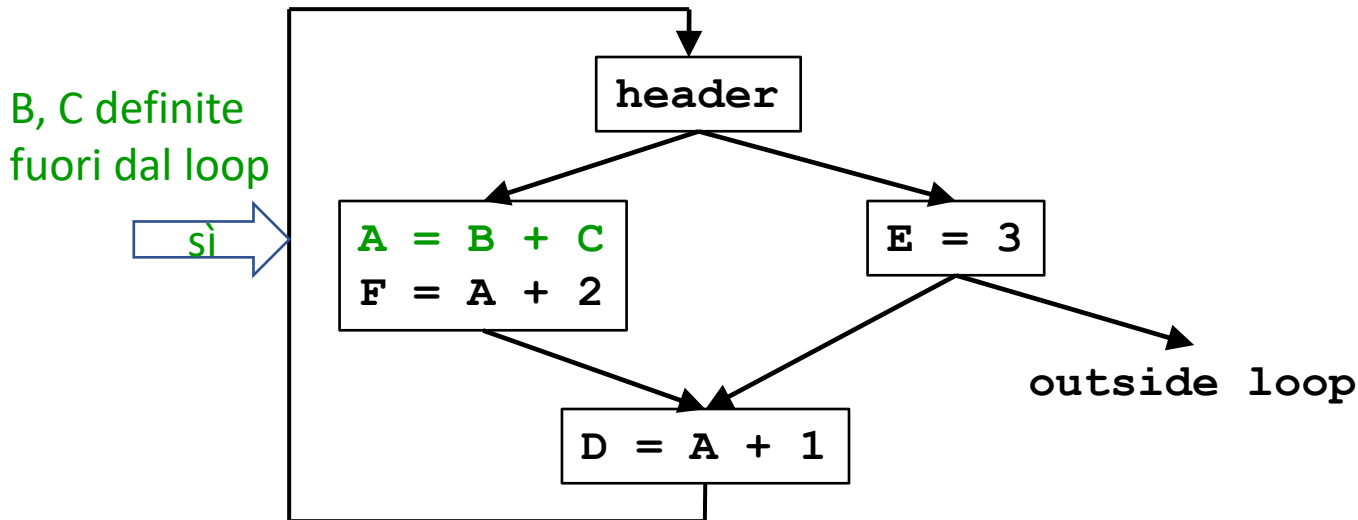
Istruzioni Loop-Invariant

- Quali istruzioni sono *loop-invariant* in questo esempio?
 - istruzioni il cui valore non cambia fintanto che il controllo rimane dentro il loop



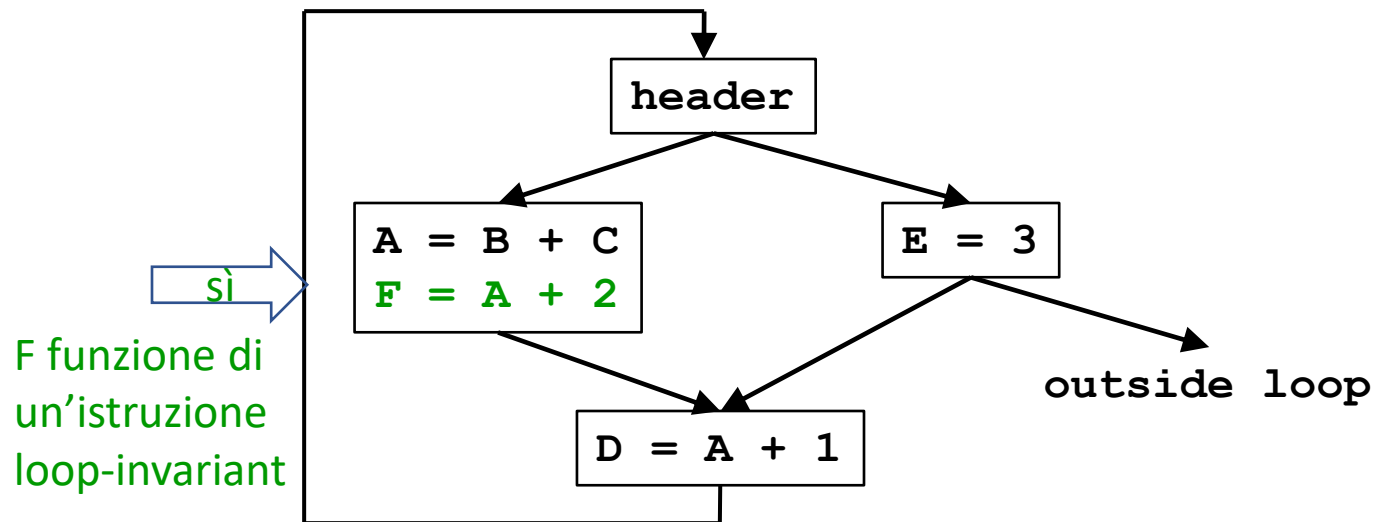
Istruzioni Loop-Invariant

- Quali istruzioni sono *loop-invariant* in questo esempio?
 - istruzioni il cui valore non cambia fintanto che il controllo rimane dentro il loop



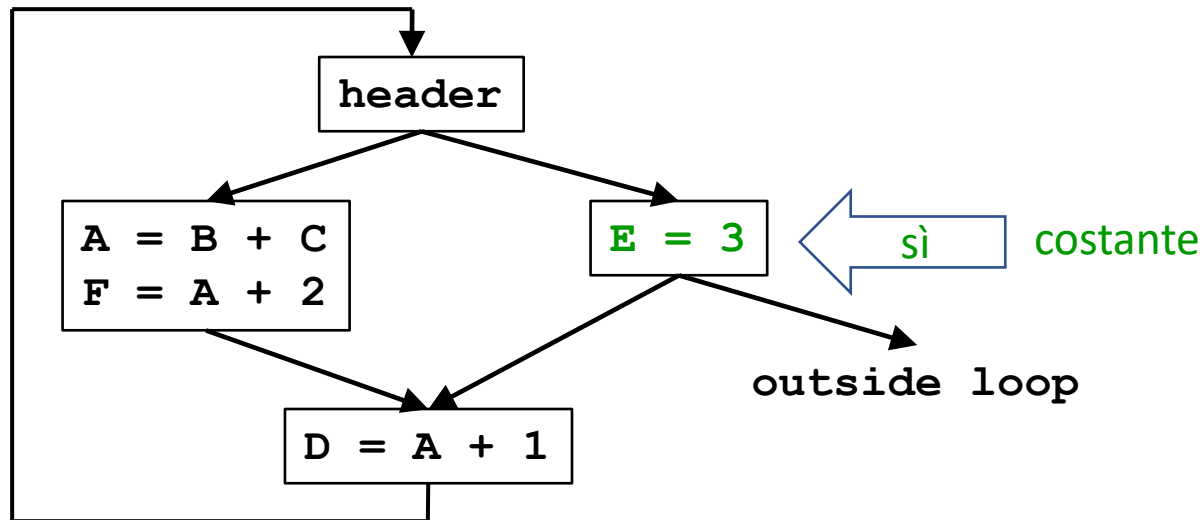
Istruzioni Loop-Invariant

- Quali istruzioni sono *loop-invariant* in questo esempio?
 - istruzioni il cui valore non cambia fintanto che il controllo rimane dentro il loop



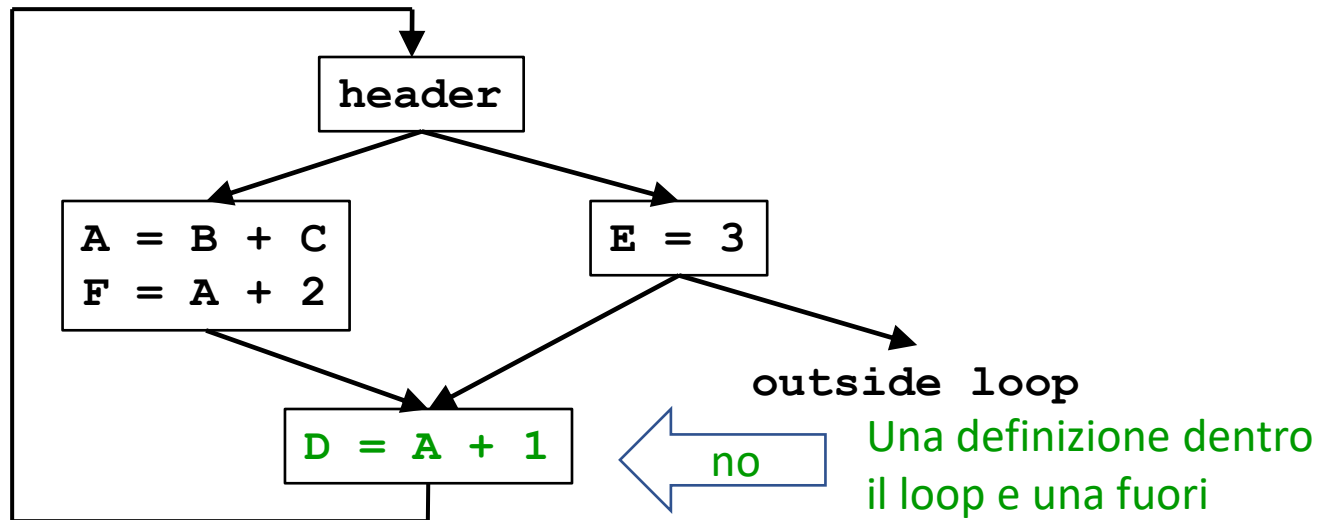
Istruzioni Loop-Invariant

- Quali istruzioni sono *loop-invariant* in questo esempio?
 - istruzioni il cui valore non cambia fintanto che il controllo rimane dentro il loop



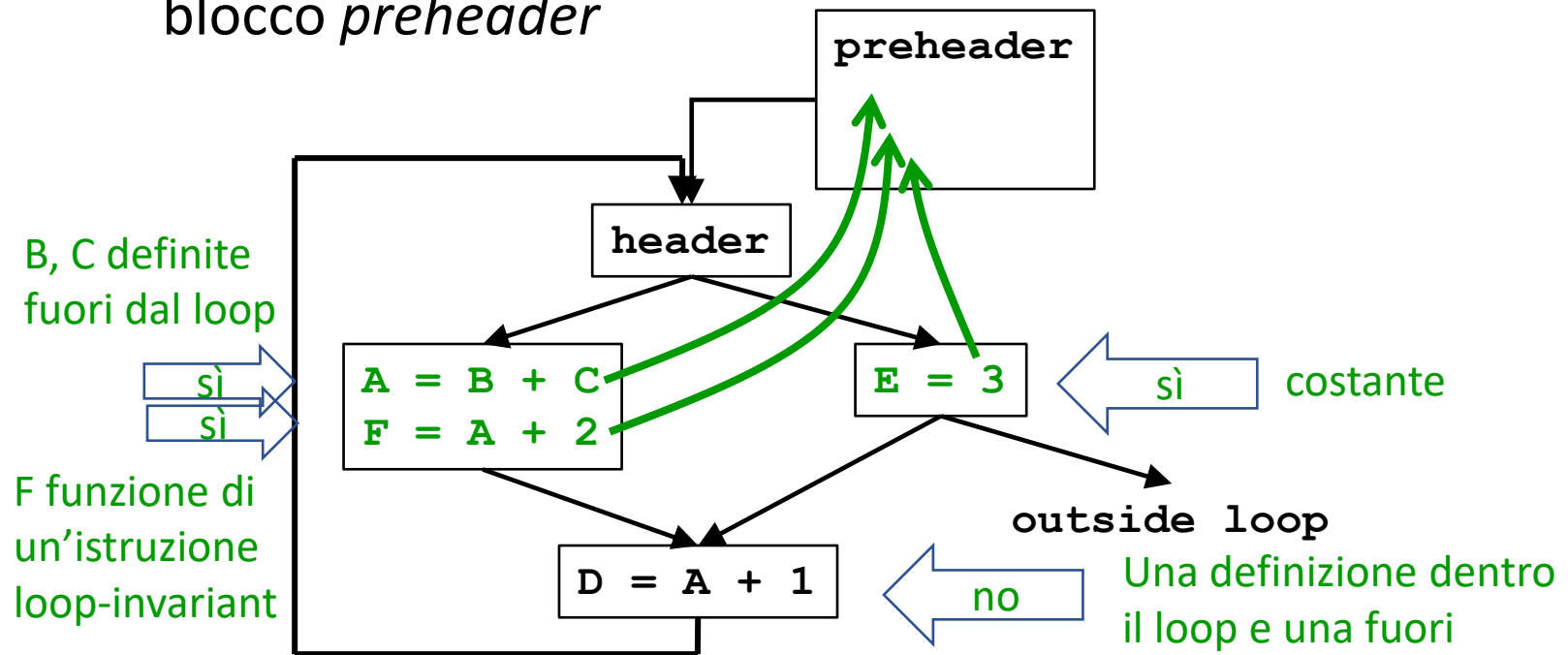
Istruzioni Loop-Invariant

- Quali istruzioni sono *loop-invariant* in questo esempio?
 - istruzioni il cui valore non cambia fintanto che il controllo rimane dentro il loop



Code Motion

- Code motion:
 - Lo spostamento delle istruzione *loop-invariant* dentro il blocco *preheader*



Algoritmo

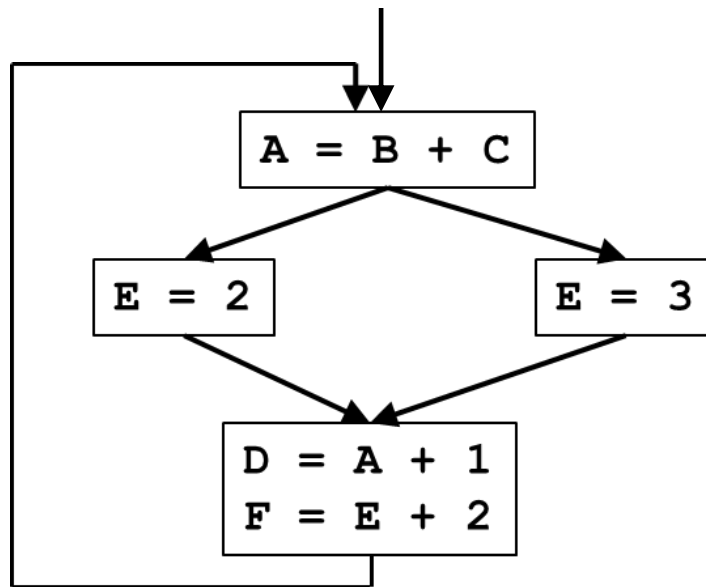
- Osservazioni
 - Istruzioni *loop-invariant*
 - Gli operandi sono definiti fuori dal loop o tramite istruzioni che sono *loop-invariant*
 - *Code motion*
 - Non tutte le istruzioni *loop-invariant* possono essere spostate nel *preheader*
- Algoritmo (tre macro fasi)
 - Troviamo le istruzioni *loop-invariant*
 - Verifichiamo che le condizioni per la *code motion* siano soddisfatte
 - Spostiamo le istruzioni

Identificare le istruzioni Loop Invariant

1. Calcolare le *reaching definitions*
2. Per ogni istruzione del tipo $A=B+C$ marcare l'istruzione come Loop-INVARIANT se:
 - Tutte le definizioni di B e C che raggiungono l'istruzione $A=B+C$ si trovano fuori dal loop
 - Se B e/o C fossero costanti?
 - **Oppure** c'è esattamente una *reaching definition* per B (e C), e si tratta di un'istruzione del loop che è stata marcata *loop-invariant*

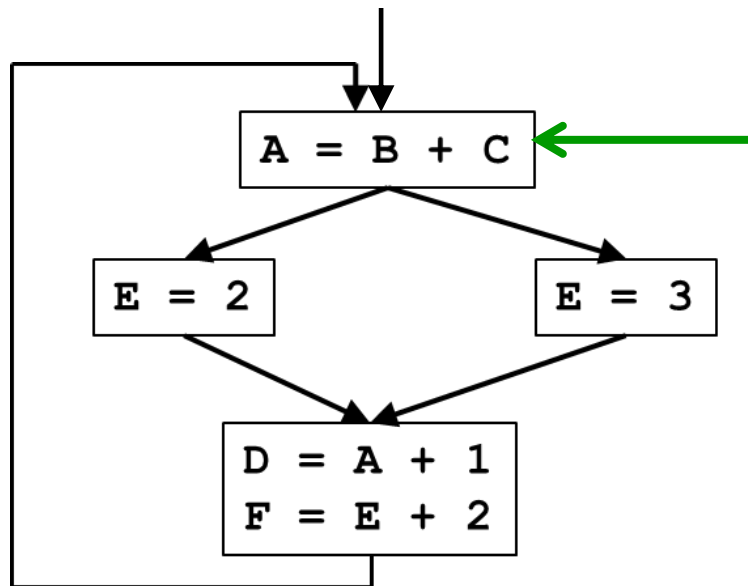
Esempio

- Quali istruzioni sono *loop-invariant*?



Esempio

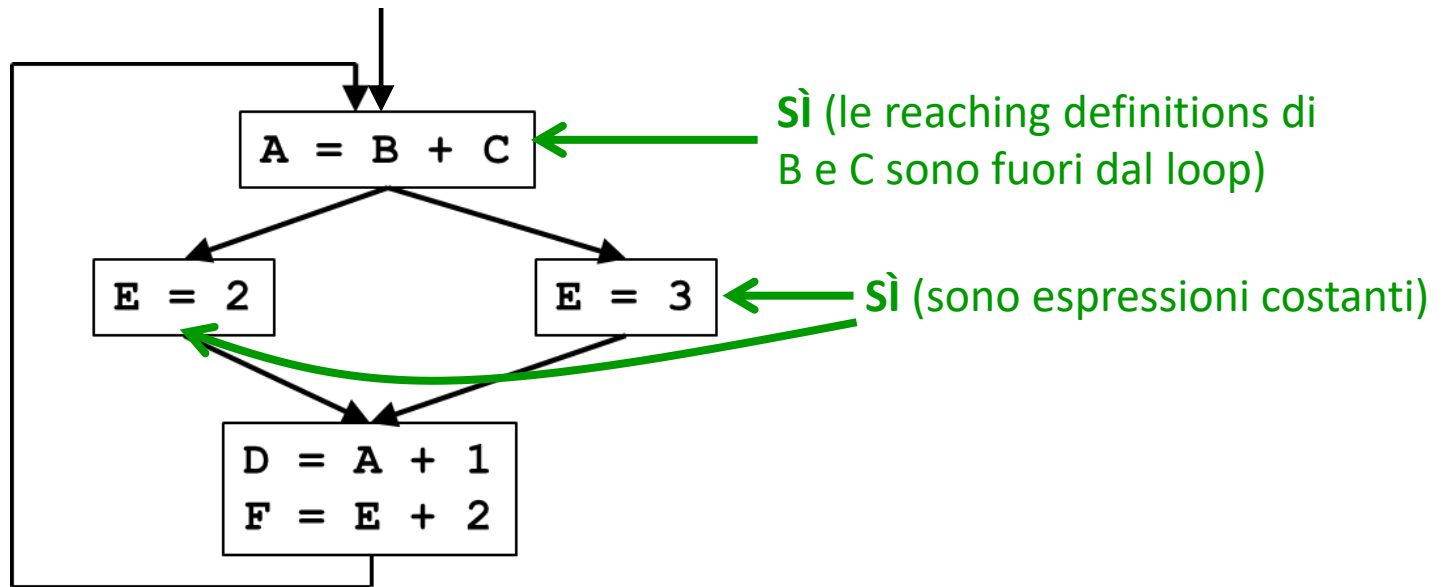
- Quali istruzioni sono *loop-invariant*?



Sì (le reaching definitions di B e C sono fuori dal loop)

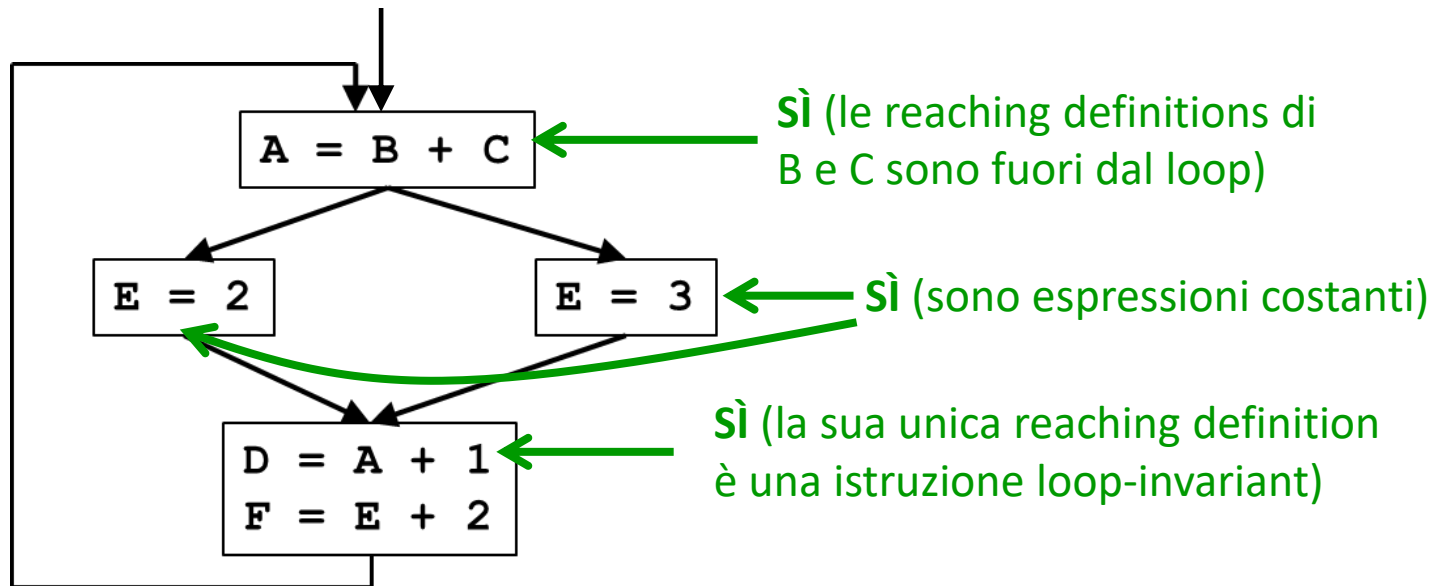
Esempio

- Quali istruzioni sono *loop-invariant*?



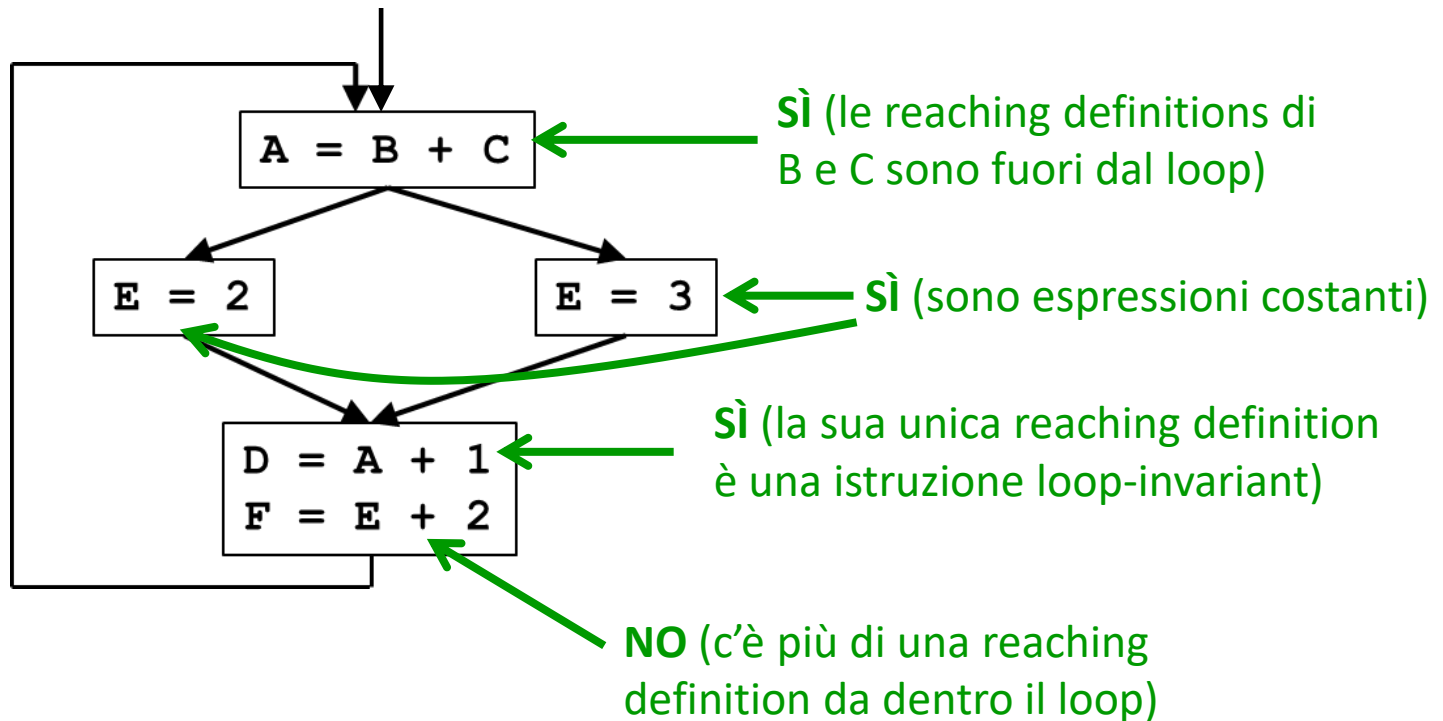
Esempio

- Quali istruzioni sono *loop-invariant*?



Esempio

- Quali istruzioni sono *loop-invariant*?



Condizioni per la Code Motion

- Condizioni generali (valide per tutte le trasformazioni):
 - **Correttezza:** Lo spostamento del codice non altera la semantica del programma
 - **Performance:** l'esecuzione del codice non è rallentata

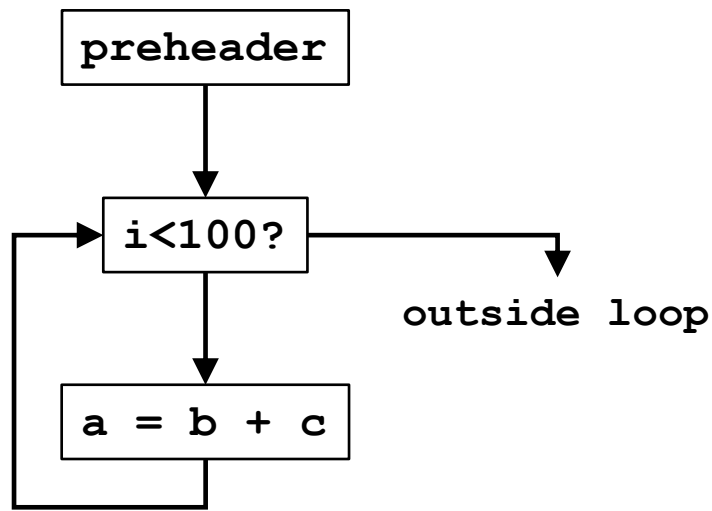
Condizioni per la Code Motion

- **Idea di base:** L'istruzione candidata per la *code motion* definisce la variabile nel suo LHS **una volta** e **per tutte** nel loop:
 - **control flow:** **una volta**?
L'istruzione (il blocco) domina tutte le uscite del loop
 - **Altre definizioni:** **per tutte**?
Non ci sono altre definizioni della variabile nel loop
 - **Altri usi:** **per tutte**?
L'istruzione (la definizione) domina tutti gli usi o non ci sono altre *reaching definitions*

Esempio

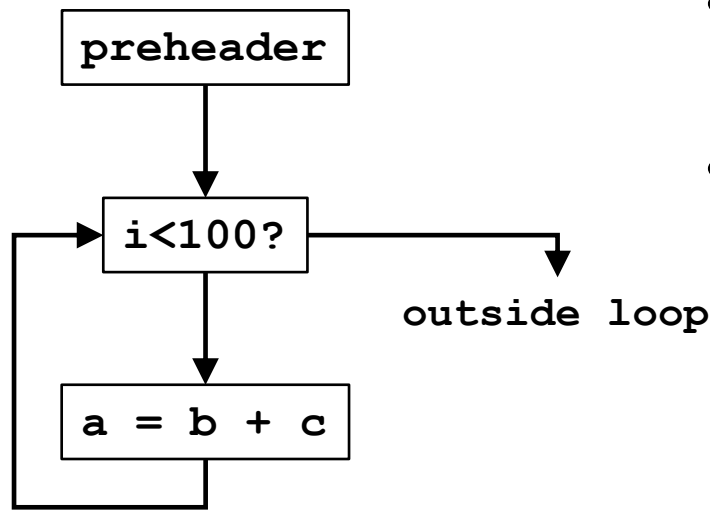
```
a = ...;  
x = ...;  
for (i = x; i < 100; i++)  
    a = b + c;
```

- L'istruzione $a = b + c$ è *loop-invariant*
- Posso procedere alla *code motion*?



Esempio

```
a = ...;  
x = ...;  
for (i = x; i < 100; i++)  
    a = b + c;
```

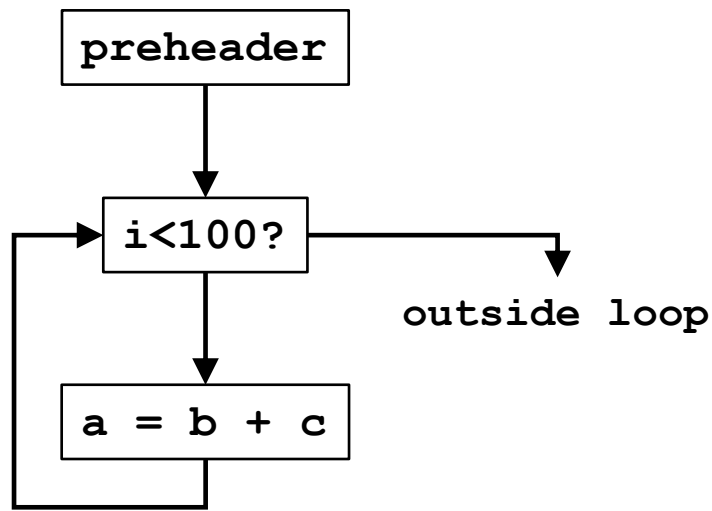


- L'istruzione $a = b + c$ è *loop-invariant*
- Ci sono altre definizioni della variabile nel loop?
- La definizione domina tutti gli usi?
- Il blocco domina tutte le uscite?

Esempio

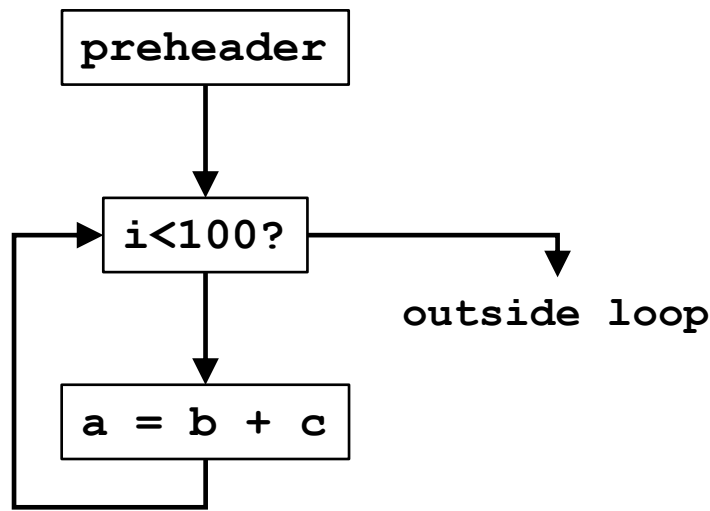
```
a = ...;  
x = ...;  
for (i = x; i < 100; i++)  
    a = b + c;
```

- Cosa succede se $x \geq 100$?



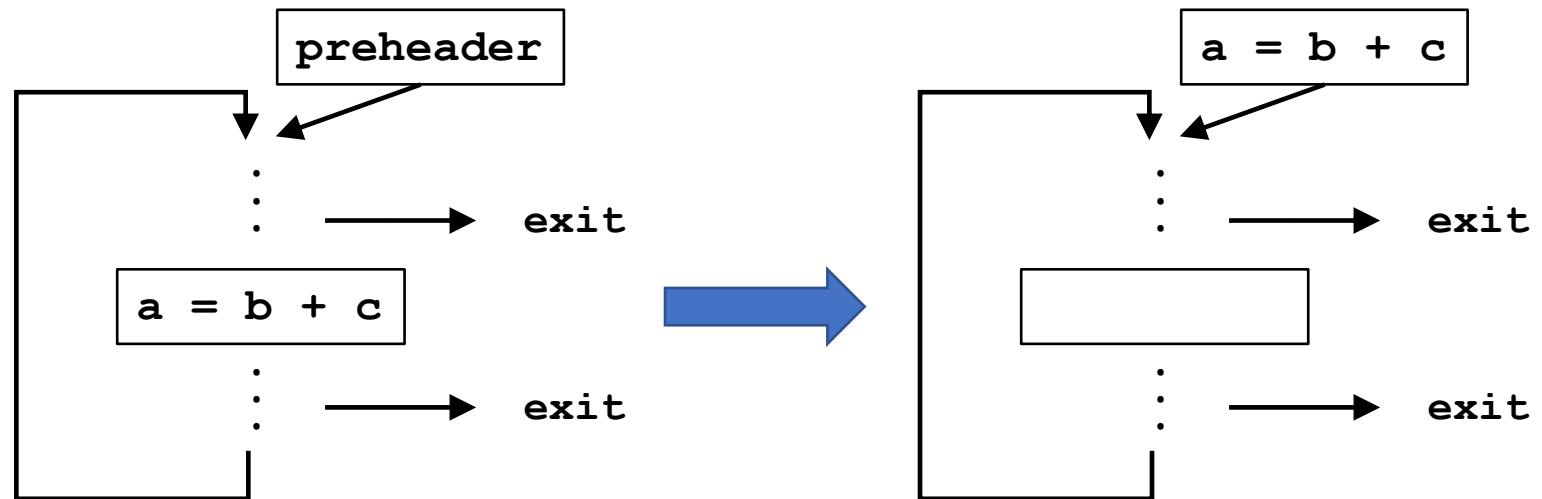
Esempio

```
a = ...;  
x = ...;  
for (i = x; i < 100; i++)  
    a = b + c;
```



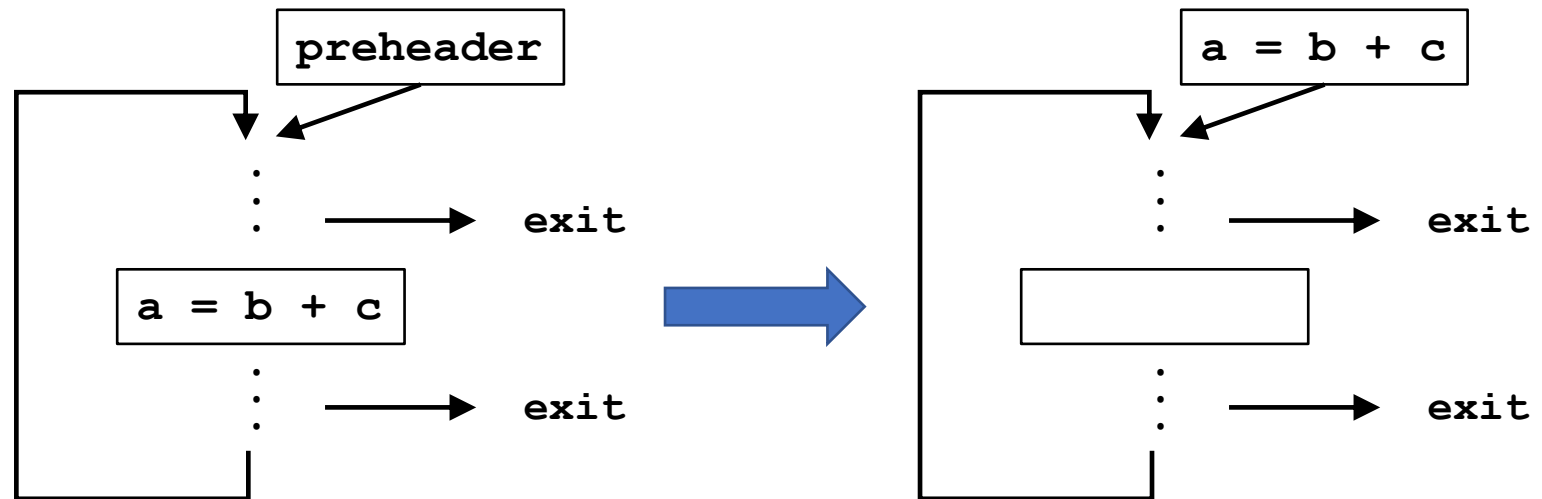
- Cosa succede se $x \geq 100$?
- Il loop non viene eseguito
- Se spostassi l'istruzione $a = b + c$ nel *preheader* cambierei la semantica del programma
 - Che porterebbe in uscita al loop la definizione di a precedente al loop

Esempio (2)



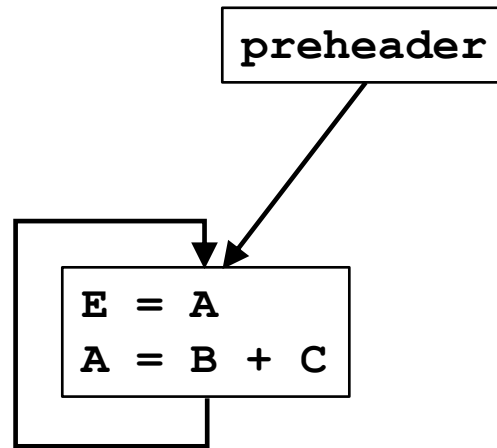
- È corretto effettuare lo spostamento del codice al *preheader*?

Esempio (2)



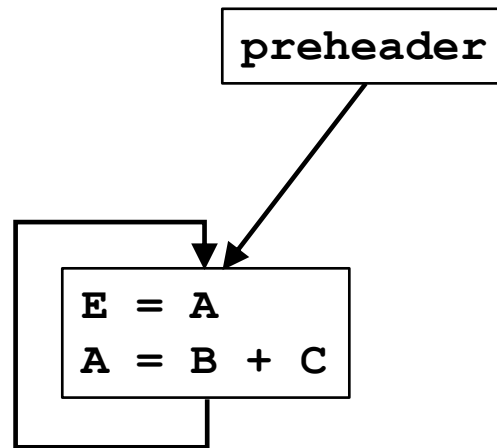
- È corretto effettuare lo spostamento del codice al *preheader*?
 - **NO**: stiamo modificando le proprietà di dominanza spostando l'istruzione prima della prima uscita

Esempio (3)



- È corretto effettuare lo spostamento del codice al *preheader*?

Esempio (3)



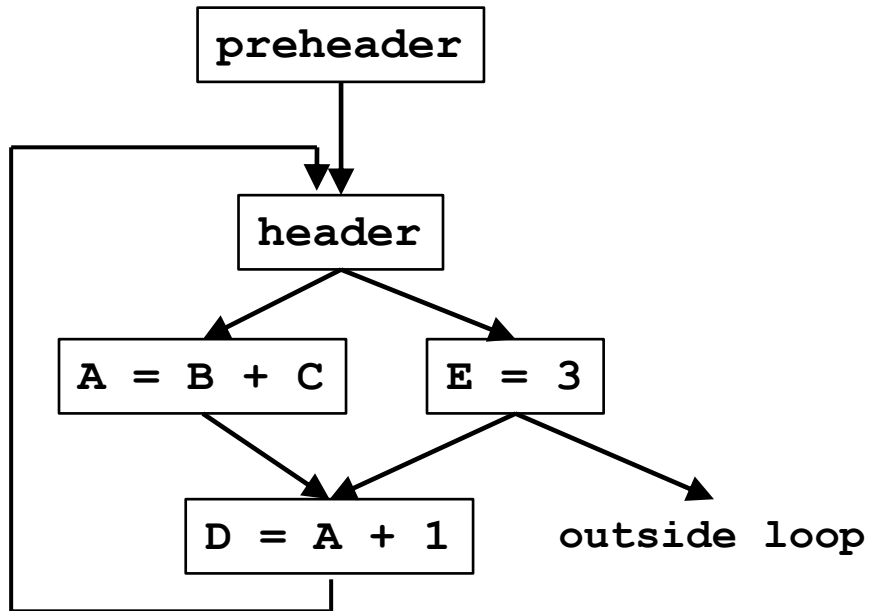
- È corretto effettuare lo spostamento del codice al *preheader*?
 - **NO**: La prima istruzione non è *loop-invariant*
 - **NO**: La seconda istruzione non domina i suoi usi

Algoritmo per la Code Motion

Dato un insieme di nodi in un loop

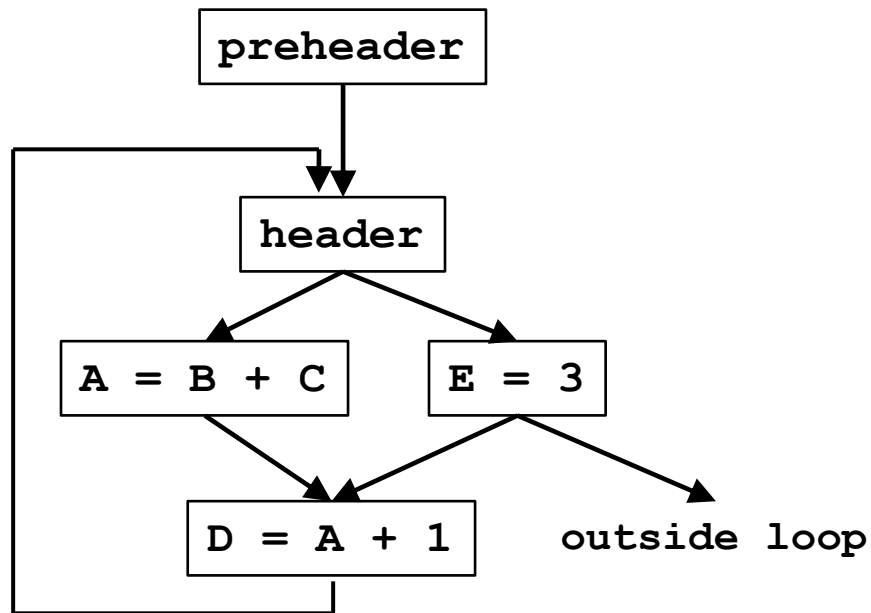
- Calcolare le *reaching definitions*
- Trovare le istruzioni *loop-invariant*
- Calcolare i dominatori (*dominance tree*)
- Trovare le uscite del loop (i successori fuori dal loop)
- Le istruzioni candidate alla *code motion*:
 - Sono *loop invariant*
 - Si trovano in blocchi che dominano tutte le uscite del loop
 - Assegnano un valore a variabili non assegnate altrove nel loop
 - Si trovano in blocchi che dominano tutti i blocchi nel loop che usano la variabile a cui si sta assegnando un valore
- Eseguire una ricerca *depth-first* dei blocchi
 - Spostare l'istruzione candidata nel *preheader* se tutte le istruzioni invarianti da cui questa dipende sono state spostate

Esempio



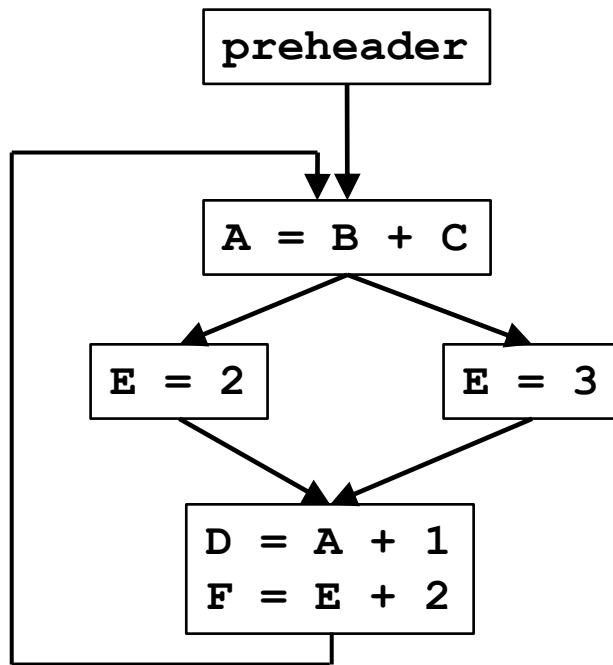
- Quali istruzioni possono essere spostate nel *preheader*?

Esempio



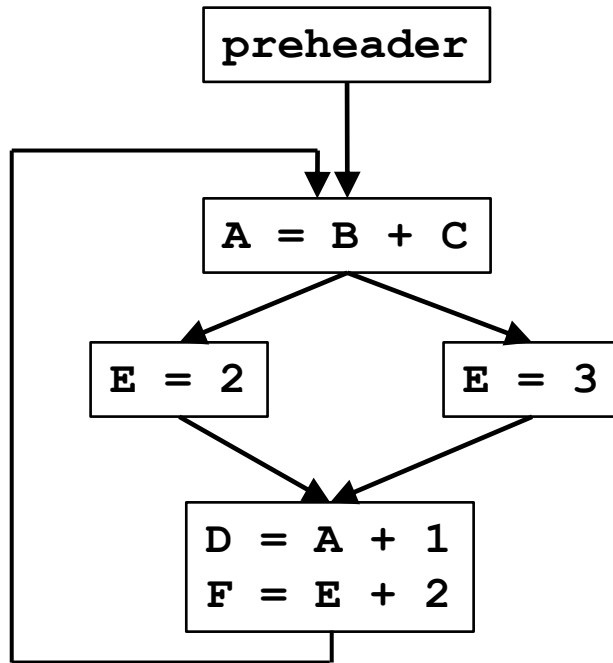
- Quali istruzioni possono essere spostate nel *preheader*?
- Solo $E = 3$
 - Unica istruzione che domina tutte le uscite

Esempio (2)



- Quali istruzioni possono essere spostate nel *preheader*?

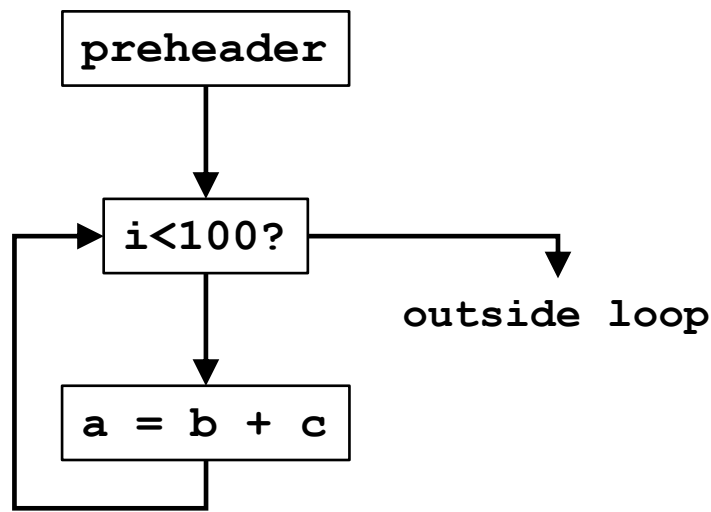
Esempio (2)



- Quali istruzioni possono essere spostate nel *preheader*?
- $A = B + C$
- $D = A + 1$
- Anche se $E=2$ e $E=3$ sono istruzioni *loop-invariant* nessuna è l'unica definizione di E
- **Ricorda:** le istruzioni devono definire la variabile **una volta e per tutte**

Possiamo essere più aggressivi?

```
a = ...;  
x = ...;  
for (i = x; i < 100; i++)  
    a = b + c;
```



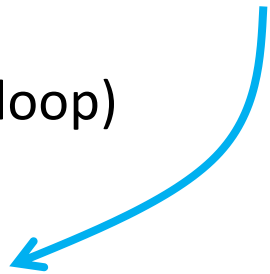
- Torniamo all'esempio di prima ($x < 100$)
- Anche se l'istruzione $a = b + c$ è *loop-invariant* non posso procedere alla *code motion*
 - Perché il blocco che ospita l'istruzione non domina tutte le uscite
- Ma cosa succederebbe se la variabile a non fosse mai usata fuori dal loop?

Algoritmo per la Code Motion

Dato un insieme di nodi in un loop

- Calcolare le *reaching definitions*
- Trovare le istruzioni *loop-invariant*
- Calcolare i dominatori (*dominance tree*)
- Trovare le uscite del loop (i successori fuori dal loop)
- Le istruzioni candidate alla *code motion*:
 - Sono *loop invariant*
 - **Si trovano in blocchi che dominano tutte le uscite del loop**
 - Assegnano un valore a variabili non assegnate altrove nel loop
 - Si trovano in blocchi che dominano tutti i blocchi nel loop che usano la variabile a cui si sta assegnando un valore
- Eseguire una ricerca *depth-first* dei blocchi
 - Spostare l'istruzione candidata nel *preheader* se tutte le istruzioni invarianti da cui questa dipende sono state spostate

Oppure la variabile definita dall'istruzione è **dead** all'uscita del loop





UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Dipartimento di Scienze Fisiche,
Informatiche e Matematiche

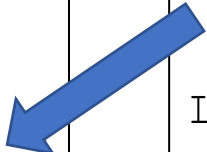
Induction Variable Elimination

Induction Variable Elimination

```
for (i = 1; i < 100; i++)  
    Arr[i] = 0;
```



```
i = 1  
L2: IF i>=100 GOTO L1  
    t1 = 4 * i  
    t2 = &Arr + t1  
    *t2 = 0  
    i = i+1  
    GOTO L2  
L1:
```



```
p1 = &A[0]+4  
p2 = 100*4  
p2 = &Arr[0]+p2  
L2: IF p1>=p2 GOTO L1  
    *p1 = 0  
    p1 = p1 + 4  
    GOTO L2  
L1:
```

- Una *induction variable* è tipicamente la variabile di controllo di un loop
 - O una sua funzione
- Si può implementare una forma di *strength reduction* per migliorare la performance

Definizioni

- Una *induction variable* **basilare (basic)** è
 - Una variabile B le cui uniche definizioni dentro il loop sono del tipo:
 - $B = B + C$ or $B = B - C$,
 - dove C è una costante o una variabile *loop-invariant*.
- Una *induction variable* è
 - Una *induction variable* basilare, oppure
 - Una variabile A definita una sola volta dentro il loop, il cui valore è una funzione lineare di una qualche *induction variable* basilare:
 $A = D + E * B$, con D ed E *loop-invariant*
- La **FAMIGLIA** di una *induction variable* basilare B è
 - L'insieme delle *induction variables* A

Induction Variable Elimination

- Sia A una induction variable nella famiglia della induction variable basilare B ($A = D + E * B$)
 - Creare una nuova variabile: A'
 - Inizializzarla nel preheader: $A' = D + E * B;$
 - Tracciare il valore di B: aggiungere dopo l'incremento di B
 $B = B + C;$
 $A' = A' + C * E;$
 - Modificare l'assegnamento ad A: $A = A'$

Induction Variable Elimination

```
i = 1
```

```
L2:
```

```
    if i > 100 goto L1
```

```
    t1 = 4 * i
```

```
    t2 = &Arr[0] + t1
```

```
    *t2 = 0
```

```
    i = i + 1
```

```
    goto L2
```

```
L1:
```

- Torniamo all'esempio di prima

- $C=1$;
- $D=\&Arr[0]$;
- $E=\text{sizeof}(int)=4$

- Sia A una induction variable nella famiglia della induction variable basilare B ($A = D + E * B$)

- Creare una nuova variabile: A'
- Inizializzarla nel preheader: $A' = D + E * B$;
- Tracciare il valore di B: aggiungere dopo l'incremento di B
 $B = B + C$;
 $A' = A' + C * E$;
- Modificare l'assegnamento ad A: $A = A'$

Induction Variable Elimination

```
i = 1
t1' = 4 * i
t2' = &Arr + t1' // Arr+4
```

```
L2:
  if i > 100 goto L1
  t1 = 4 * i
  t2 = &Arr[0] + t1
  t1 = t1'
  t2 = t2'
  *t2 = 0
  i = i + 1
  t1' = t1' + 1 * 4
  t2' = t2' + 1 * 4
  goto L2
L1:
```

- Torniamo all'esempio di prima

- $C=1;$
- $D=\&Arr[0];$
- $E=\text{sizeof}(int)=4$

- Sia A una induction variable nella famiglia della induction variable basilare B ($A = D + E * B$)

- Creare una nuova variabile: A'
- Inizializzarla nel preheader: $A' = D + E * B;$
- Tracciare il valore di B: aggiungere dopo l'incremento di B
 $B = B + C;$
 $A' = A' + C * E;$
- Modificare l'assegnamento ad A: $A = A'$

Induction Variable Elimination

- NOTA: Le istruzioni non necessarie possono essere eliminate:
 - copy propagation
 - dead code elimination
- 3. Rimozione delle induction variables basilari
 - Possiamo eliminare le induction variables basilari usate solo per
 - Calcolare altre induction variables e effettuare i test di controllo del loop
 - Algoritmo:
 - Selezionare una induction variable A nella famiglia di B ($A = D + E * B$).
 - Rimpiazzare le istruzioni di confronto del tipo

```
if B > X GOTO L1
```


con

```
if (A' > D + E * X) GOTO L1
```

 (per $E > 0$)
 - Se B è live a qualsiasi uscita del loop, ricalcolarla a partire da A'
 - $B = (A' - D) / E$

Induction Variable Elimination

```
i = 1
t1' = 4 * i
t2' = &Arr + t1'
```

```
L2:
    if i > 100 goto L1
t1 = 4 * i
t2 = &Arr[0] + t1
t1 = t1'
    t2 = t2'
    *t2 = 0
i = i + 1
t1' = t1' + 1 * 4
    t2' = t2' + 1 * 4
    goto L2
L1:
```

- Torniamo all'esempio di prima

- DEAD CODE ELIMINATION

- NOTA: Le istruzioni non necessarie possono essere eliminate:
 - copy propagation
 - dead code elimination
- 3. Rimozione delle induction variables basilari
 - Possiamo eliminare le induction variables basilari usate solo per
 - Calcolare altre induction variables e effettuare i test di controllo del loop
 - Algoritmo:
 - Selezionare una induction variable A nella famiglia di B ($A = D + E * B$).
 - Rimpiazzare le istruzioni di confronto del tipo
if B > X GOTO L1
con
if ($A' > D + E * X$) GOTO L1 (per $E > 0$)
 - Se B è live a qualsiasi uscita del loop, ricalcolarla a partire da A'
 - $B = (A' - D) / E$

Induction Variable Elimination

```
i = 1
t1' = 4 * i
t2' = &Arr + t1'
```

```
L2:
  if i > 100 goto L1
  t2 = t2'
  *t2' = 0
  t2' = t2' + 1 * 4
  goto L2
L1:
```

- Torniamo all'esempio di prima

- COPY PROPAGATION

- NOTA: Le istruzioni non necessarie possono essere eliminate:
 - copy propagation
 - dead code elimination
- 3. Rimozione delle induction variables basilari
 - Possiamo eliminare le induction variables basilari usate solo per
 - Calcolare altre induction variables e effettuare i test di controllo del loop
 - Algoritmo:
 - Selezionare una induction variable A nella famiglia di B ($A = D + E * B$).
 - Rimpiazzare le istruzioni di confronto del tipo

```
if B > X GOTO L1
```

con

```
if (A' > D + E * X) GOTO L1      (per E > 0)
```
 - Se B è live a qualsiasi uscita del loop, ricalcolarla a partire da A'
 - $B = (A' - D) / E$

Induction Variable Elimination

```
i = 1
t1' = 4 * i
t2' = &Arr + t1'
t3 = 4 * 100
t3 = &Arr + t3
```

```
L2:
  if t2' > t3 goto L1
  t2 = t2'
  *t2' = 0
  t2' = t2' + 1 * 4
  goto L2
L1:
```

- Torniamo all'esempio di prima

- INDUCTION VARIABLE ELIMINATION
 - $X = 100$

- NOTA: Le istruzioni non necessarie possono essere eliminate:
 - copy propagation
 - dead code elimination
- 3. Rimozione delle induction variables basilari
 - Possiamo eliminare le induction variables basilari usate solo per
 - Calcolare altre induction variables e effettuare i test di controllo del loop
 - Algoritmo:
 - Selezionare una induction variable A nella famiglia di B ($A = D + E * B$).
 - Rimpiazzare le istruzioni di confronto del tipo

```
if B > X GOTO L1
```

con

```
if (A' > D + E * X) GOTO L1      (per E > 0)
```
 - Se B è live a qualsiasi uscita del loop, ricalcolarla a partire da A'
 - $B = (A' - D) / E$

Induction Variables Basilari

Una variabile B le cui uniche definizioni dentro il loop sono del tipo:

$B=B+C$ or $B=B-C$, dove C è una costante o una variabile *loop-invariant*.

- Esempio:

```
k = 0;
for (i = 0; i < n; i++) {
    k = k + 3;
    ... = m;
    if (x < y)
        k = k + 4;
    if (a < b)
        m = 2 * k;
    k = k - 2;
    ... = m;
```

Si possono facilmente determinare
scorrendo il loop

Ogni iterazione può eseguire un
numero di incrementi/decrementi
differenti!!

Induction Variables Basilari

Una variabile B le cui uniche definizioni dentro il loop sono del tipo:

$B=B+C$ or $B=B-C$, dove C è una costante o una variabile *loop-invariant*.

- Esempio:

```
k = 0;
for (i = 0; i < n; i++) {
    k = k + 3;
    ... = m;
    if (x < y)
        k = k + 4;
    if (a < b)
        m = 2 * k;
    k = k - 2;
    ... = m;
```

Si possono facilmente determinare
scorrendo il loop

IV basilari: i, k

Altre IV: $m = 0 + 2k$ (famiglia di k)

Ogni iterazione può eseguire un
numero di incrementi/decrementi
differenti!!

Determinare le famiglie di IV

- Sia B una IV basilare
 - Trovare tutte le IV A nella famiglia di B:
 - $A = D + E * B$
(dove B si riferisce al valore di B al tempo della definizione)
- A è una IV nella famiglia di B:
 - Se A ha un solo assegnamento nel loop L, e l'assegnamento è del tipo:
 - $A = B * c$
 - $A = c * B$
 - $A = B / c$ (se A è un numero reale)
 - $A = B + c$
 - $A = c + B$
 - $A = B - c$
 - $A = c - B$
- OPPURE (prossima pagina)

Finding Induction Variable Families (continued)

- D è una IV nella famiglia di B ($D = c1 * B + c2$)
 - Se A ha un solo assegnamento nel L, e l'assegnamento è del tipo:
 - $A = D * c$
 - $A = c * D$
 - $A = D / c$ (se A è un numero reale)
 - $A = D + c$
 - $A = c + D$
 - $A = D - c$
 - $A = c - D$
 - Se nessuna definizione di D fuori dal loop L raggiunge l'assegnamento ad A
 - Se tra l'assegnamento a D nel loop L e l'assegnamento ad A non ci sono (ri)definizioni di B

Esempio

L2:

```
if i > 100 goto L1
t2 = t1 + 10
t1 = 4 * i
t3 = t1 * 8
i = i + 1
goto L2
```

L1:

- Condizione C1
 - A ha un solo assegnamento nel loop L del tipo $A = B * c, c * B, B + c, c + B$, ecc
- Condizione C2
 - A è nella famiglia di B se $D = c_1 + c_2 * B$ e:
 - Regola 1: A ha un solo assegnamento nel loop L del tipo $A = D * c, c * D, D + c, c + D$, ecc
 - Regola 2: Nessuna definizione di D esterna a L raggiunge l'assegnamento ad A
 - Regola 3: Tra l'assegnamento a D nel loop L e l'assegnamento ad A non ci sono (ri)definizioni di B

Esempio

L2:

if i > 100 goto L1	i è una IV basilare?
t2 = t1 + 10	t2 appartiene alla famiglia di i ?
t1 = 4 * i	t1 appartiene alla famiglia di i ?
t3 = t1 * 8	t3 appartiene alla famiglia di i ?
i = i + 1	
goto L2	

L1:

- Condizione C1
 - A ha un solo assegnamento nel loop L del tipo $A = B * c, c * B, B + c, c + B$, ecc
- Condizione C2
 - A è nella famiglia di B se $D = c_1 + c_2 * B$ e:
 - Regola 1: A ha un solo assegnamento nel loop L del tipo $A = D * c, c * D, D + c, c + D$, ecc
 - Regola 2: Nessuna definizione di D esterna a L raggiunge l'assegnamento ad A
 - Regola 3: Tra l'assegnamento a D nel loop L e l'assegnamento ad A non ci sono (ri)definizioni di B

Esempio

L2:

```
if i > 100 goto L1
t2 = t1 + 10
t1 = 4 * i
t3 = t1 * 8
i = i + 1
goto L2
```

L1:

i è una IV basilare?

Sì

t2 appartiene alla famiglia di **i**?

NO (regola 2)

t1 appartiene alla famiglia di **i**?

Sì (per C1)

t3 appartiene alla famiglia di **i**?

Sì (per C2 con
A=t3; D=t1; B=i)

- Condizione C1
 - A ha un solo assegnamento nel loop L del tipo $A = B * c, c * B, B + c, c + B$, ecc
- Condizione C2
 - A è nella famiglia di B se $D = c_1 + c_2 * B$ e:
 - Regola 1: A ha un solo assegnamento nel loop L del tipo $A = D * c, c * D, D + c, c + D$, ecc
 - Regola 2: Nessuna definizione di D esterna a L raggiunge l'assegnamento ad A
 - Regola 3: Tra l'assegnamento a D nel loop L e l'assegnamento ad A non ci sono (ri)definizioni di B

Esempio (2)

L3:

```
if i > 100 goto L1
t1 = 4 * i
if t1 < 50 goto L2
i = i + 2
```

i è una IV basilare?

t2 appartiene alla famiglia di **i**?

t1 appartiene alla famiglia di **i**?

L2:

```
t2 = t1 + 10
i = i + 1
goto L3
```

L1:

- Condizione C1
 - A ha un solo assegnamento nel loop L del tipo $A = B * c, c * B, B + c, c + B$, ecc
- Condizione C2
 - A è nella famiglia di B se $D = c_1 + c_2 * B$ e:
 - Regola 1: A ha un solo assegnamento nel loop L del tipo $A = D * c, c * D, D + c, c + D$, ecc
 - Regola 2: Nessuna definizione di D esterna a L raggiunge l'assegnamento ad A
 - Regola 3: Tra l'assegnamento a D nel loop L e l'assegnamento ad A non ci sono (ri)definizioni di B

Esempio (2)

L3:

```
if i > 100 goto L1
t1 = 4 * i
if t1 < 50 goto L2
i = i + 2
```

L2:

```
t2 = t1 + 10
i = i + 1
goto L3
```

L1:

i è una IV basilare?

Sì

t2 appartiene alla famiglia di **i**? NO (regola 3)

t1 appartiene alla famiglia di **i**? Sì (per C1)

- Condizione C1
 - A ha un solo assegnamento nel loop L del tipo $A = B * c, c * B, B + c, c + B$, ecc
- Condizione C2
 - A è nella famiglia di B se $D = c_1 + c_2 * B$ e:
 - Regola 1: A ha un solo assegnamento nel loop L del tipo $A = D * c, c * D, D + c, c + D$, ecc
 - Regola 2: Nessuna definizione di D esterna a L raggiunge l'assegnamento ad A
 - Regola 3: Tra l'assegnamento a D nel loop L e l'assegnamento ad A non ci sono (ri)definizioni di B